

지원 포트폴리오

고객의 불편에서 시작해 우선순위를 정하고, backend·AI·핵심 frontend를 직접 구현해 실제 고객이 결제하는 제품을 만들었습니다.

제품 출시·AI 작업 전달·결제 상태·실시간 상담에서 직접 설계·구현한 범위와 팀과 함께한 범위를 네 사례로 나눴습니다.

Cases · 4

01 고객의 불편을 찾아 유료 제품으로 만들었습니다

고객 문제와 제품 우선순위를 정하고, 필요한 backend·AI·핵심 frontend를 직접 구현해 팀과 유료 제품으로 운영했습니다.

02 AI 작업이 중복되거나 늦어져도 최신 결과만 반영했습니다

제품 원장과 AI 실행을 분리하고, 전달 실패와 worker 중단을 복구할 수 있는 상태로 다뤘습니다.

03 Stripe 선결제부터 취소·환불까지 상태 흐름을 만들었습니다

실제로 구현한 Checkout·Webhook·취소·환불을 먼저 보여주고, 이후 보완할 복구 구조는 별도 설계안으로 뒀습니다.

04 중간 전사와 확정 문맥이 충돌하지 않도록 상담 backend를 설계했습니다

STT partial·final·보정 event의 순서와 WebSocket session·비동기 task lifecycle을 함께 설계했습니다.

PM으로 제품의 방향을 정했고, 엔지니어로 고객이 결제하는 제품을 만들었습니다

Vision AI와 Product Manager 경험을 제품 판단의 기반으로 삼고, Backend Engineer로 역할을 확장해 제품의 상태·데이터·실패 경계를 직접 구현하고 운영해 왔습니다.

제품 방향 생성형 AI 커머스 제품의 프로토타입에서 v1.0까지 기능 범위와 출시 우선순위를 정하고 첫 정식 버전 출시를 PM으로 이끌었습니다. PRODUCT AI	01
직접 구현 AI 콘텐츠 제품에서 backend·AI·핵심 frontend를 직접 만들고 기획·QA·마케팅과 제품 운영을 리드했습니다. PRODUCT BACKEND AI	02
유료 운영 제품 판단과 구현을 출시 이후의 운영까지 연결해 팀과 실제 고객이 결제하는 제품으로 전환했습니다. PRODUCT BACKEND OPERATIONS	03

SellerCanvas와 Thready는 서로 다른 시기와 역할의 사례입니다. 두 제품의 결과를 하나의 프로젝트나 개인 단독 성과로 합치지 않습니다.

01 고객의 불편을 찾아 유료 제품으로 만들었습니다

고객 문제와 제품 우선순위를 정하고, 필요한 backend·AI·핵심 frontend를 직접 구현해 팀과 유료 제품으로 운영했습니다.

단일 시스템 사례

PRODUCT

BACKEND

AI

OPERATIONS

운영 중

상황 · Threads 콘텐츠 제작과 성과 판단을 돕는 AI 콘텐츠 제품 Thready의 초기 제품화와 운영 사례입니다.

문제

고객에게는 글을 생성하는 기능만이 아니라 어떤 콘텐츠가 반응을 얻는지 판단하고 반복할 기준이 필요했습니다. 초기 prototype은 빠른 검증에는 적합했지만 실제 사용자 운영을 이어갈 backend 구조와 생성 품질 기준이 충분히 분리되지 않았습니다.

대처 방법

- 고객 불편을 기능·실험·품질 기준으로 나누고 기획·QA·마케팅과 제품 우선순위를 정했습니다.
- 기존 사용자 흐름을 유지하면서 FastAPI backend를 재구축하고 생성·가져오기·예약·발행·관리 화면을 직접 구현했습니다.
- 생성 품질을 typed prompt·평가 loop·관측 logging으로 비교 가능한 대상으로 만들었습니다.

결과

팀과 실제 고객이 결제하는 제품으로 전환해 운영하고 있습니다. 정확한 매출액이나 단일 기능과 매출 사이의 직접 인과는 공개하지 않습니다.

고객 문제에서 유료 운영까지

설계 흐름

고객 문제를 제품 판단으로 바꾸고, 구현부터 출시·운영까지 실제로 책임진 흐름을 한 줄로 정리했습니다.



고객 문제를 제품 판단으로 바꾸고 필요한 기술 경계를 직접 구현한 뒤, 팀과 출시·운영해 실제 고객 결제까지 연결한 흐름입니다.

기여 내용

해결 과정에서 맡은 기술 판단과 구현 범위입니다.

제품 문제

고객의 문제를 prompt 개선 하나로 축소하지 않고 제품 흐름·품질 기준·운영 구조로 함께 다뤄야 했습니다.

제품 판단

콘텐츠 제작과 반응 판단의 불편을 기능·실험·품질 기준으로 분리해 출시 우선순위를 정했습니다.

직접 구현

Product backend

FastAPI API·domain·transaction·migration과 release 이후 운영을 맡았습니다.

AI product system

생성 pipeline·typed prompt·평가 loop·관측 logging을 구현했습니다.

Product surface

Next.js 생성·가져오기·예약·발행·dashboard·admin 핵심 workflow를 구현했습니다.

협업 범위

제품 운영과 직접 구현 범위는 리드했지만 기획·QA·마케팅의 팀 기여와 제품·매출 outcome을 개인 단독 성과로 귀속하지 않습니다.

담당 범위

리드 **출시부터 유료 운영까지**

기획·QA·마케팅과 제품 운영을 리드하고 팀과 실제 고객이 결제하는 제품으로 전환했습니다.

직접 소유 **Backend·AI·핵심 frontend**

제품을 완성하는 데 필요한 backend·AI·핵심 frontend 사용자·관리 workflow를 직접 구현했습니다.

WHATSSUB JD FIT
아이디어와 요구사항의 빠른 prototype·출시 Backend뿐 아니라 제품에 필요한 web·data 기능 구현 설계부터 출시·운영까지 제품 전체 흐름을 책임진 경험
마이데이터 경험을 대신할 수는 없지만, 새로운 제품을 고객이 쓰고 결제하는 서비스로 만든 경험은 왓섭의 신규 제품 출시와 맞닿아 있습니다.

AI 작업이 중복되거나 늦어져도 최신 결과만 반영했습니다

제품 원장과 AI 실행을 분리하고, 전달 실패와 worker 중단을 복구할 수 있는 상태로 다뤘습니다.

[단일 시스템 사례](#)[PRODUCT](#)[BACKEND](#)[AI](#)[OPERATIONS](#)[STG·Prod 운영](#)

상황 · 제품 정책·원장과 AI 생성 lifecycle·실행 상태를 독립 FastAPI application·DB로 분리해 STG·Prod에서 운영한 Thready 사례입니다.

문제

application을 분리하면 product backend의 변경은 성공했지만 전달 기록이 사라지거나, worker 중단 뒤 같은 event가 다시 실행되거나, 늦게 도착한 이전 event가 최신 상태를 덮는 failure mode가 생깁니다.

대처 방법

- 제품 상태 변경과 Outbox 기록만 같은 transaction으로 묶고, 시간이 오래 걸리는 AI 실행은 transaction 밖으로 분리했습니다.
- relay worker는 lease와 attempt token으로 실행 권한을 확인하고, 중단된 작업은 lease 만료 뒤 다시 점유하게 했습니다.
- delivery version fence·먹등 consumer·제한된 retry·terminal failure 보존으로 지연·중복·역순 event를 통제했습니다.

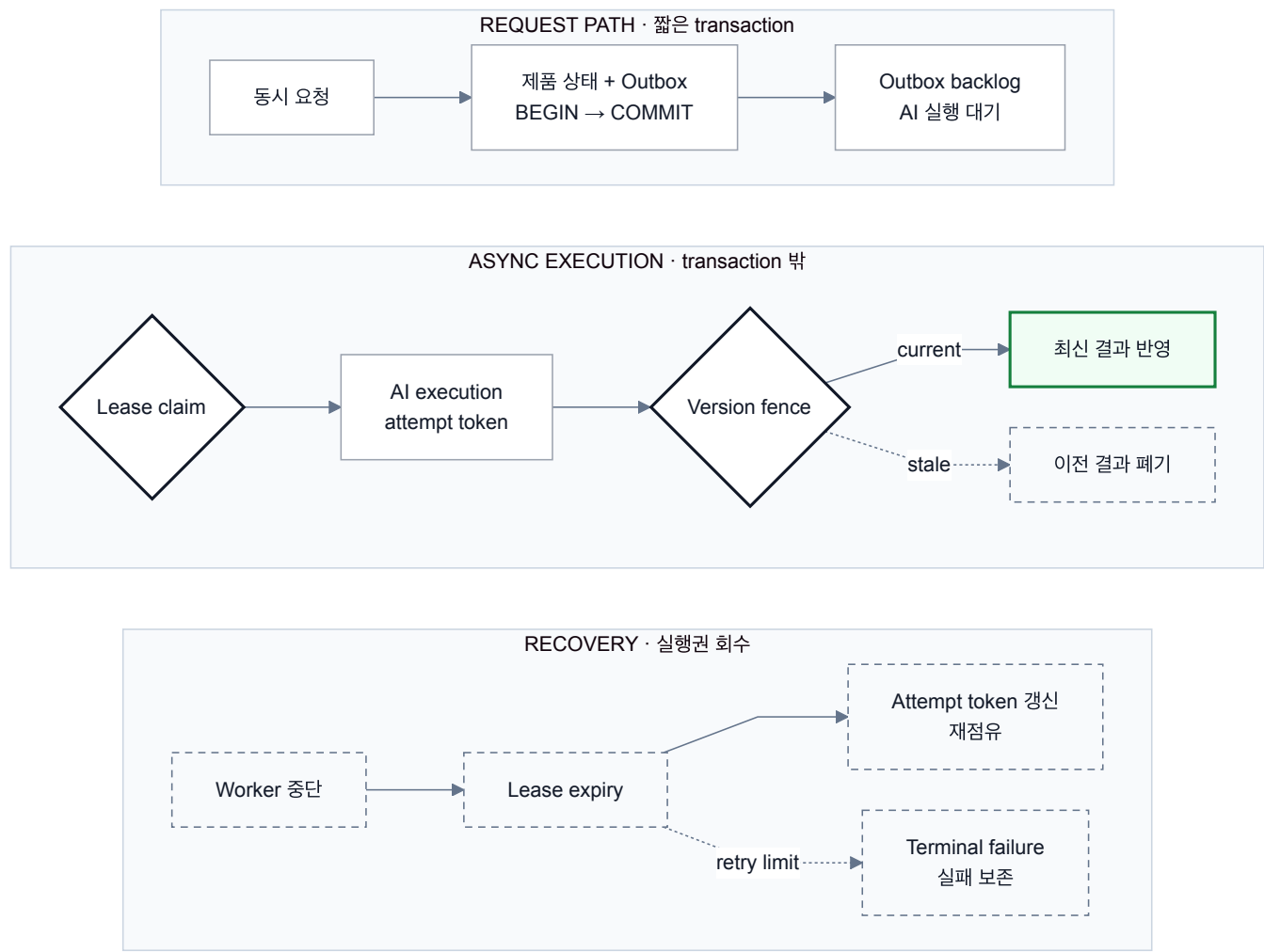
결과

AI 장애와 worker 중단을 특별한 예외가 아니라 관측·재시도·재점유 가능한 운영 상태로 남기고, 늦은 event가 최신 원장 상태를 되돌리지 못하게 했습니다.

Transaction은 제품 상태와 전달 의도까지만 묶었습니다

설계 흐름

요청 transaction이 끝나는 지점과 Outbox 적체·worker 재점유·최신 결과 반영 경계를 한 흐름으로 정리했습니다.



API transaction은 제품 상태와 전달 의도를 기록한 뒤 끝나고, AI 실행과 재시도는 transaction 밖에서 이어집니다.

트래픽이 늘어날 때

동작 · 검증 지점

현재 구현으로 설명할 수 있는 동작과, 요청량이 커질 때 별도 부하 검증이 필요한 지점을 나눴습니다.

상황	Transaction·worker 동작	부하 검증 항목
요청 급증	요청마다 제품 상태와 Outbox 기록까지만 commit하고, AI 실행은 API transaction 밖에서 처리합니다.	Outbox backlog age · commit latency
Worker 경합	lease와 attempt token으로 현재 실행권을 구분해 같은 작업의 이전 실행이 완료 상태를 덮지 못하게 합니다.	claim latency · DB lock wait

Worker 중단	완료되지 않은 lease가 만료되면 실행권을 회수하고 새 attempt token으로 작업을 다시 점유합니다.	lease expiry · retry rate
이전 결과 지연	delivery version fence와 먹등 consumer가 현재 version을 확인해 늦거나 중복된 결과를 반영하지 않습니다.	stale event · terminal failure

기여 내용

해결 과정에서 많은 기술 판단과 구현 범위입니다.

상황

두 DB를 하나의 transaction으로 묶을 수 없으므로 전달 실패를 숨기지 않고 복구 가능한 상태로 남겨야 했습니다.

TRANSACTION 경계

제품 상태와 전달 의도까지만 같은 transaction으로 commit하고, 생성 lifecycle·실행 상태는 AI application이 별도로 소유하게 했습니다.

대처 방법

짧은 원장 transaction

제품 상태 변경과 Outbox 기록을 함께 commit하고, AI 실행이 API transaction을 오래 잡지 않도록 분리했습니다.

실행권 회수

lease·attempt token으로 작업 실행 권한을 확인하고 중단된 작업을 재점유할 수 있게 했습니다.

최신 결과 보호

delivery version fence·먹등 consumer·terminal failure 보존으로 늦거나 중복된 event를 흡수했습니다.

운영 한계

Exactly-once·무중단·무유실은 주장하지 않습니다. 최대 재시도 뒤 실패를 보존하고 운영자가 확인하는 at-least-once delivery입니다.

담당 범위

직접 소유 **AI application·data ownership**

제품 원장과 AI 생성 lifecycle·실행 상태를 독립 application·DB로 분리해 운영했습니다.

직접 소유 **Durable delivery protocol**

Transactional Outbox·lease·attempt token·retry·version fence·역등 consumer·terminal failure를 구현했습니다.

WHATSSUB JD FIT

Product core와 AI execution의 service·data ownership

분산 transaction 대신 durable intent와 at-least-once delivery를 선택한 판단

Worker 중단·중복·지연·역순 event를 복구 가능한 상태로 운영

금융 데이터 경험은 아니지만, 서로 다른 application·DB 사이의 정합성과 복구를 직접 설계한 경험입니다.

Stripe 선결제부터 취소·환불까지 상태 흐름을 만들었습니다

실제로 구현한 Checkout·Webhook·취소·환불을 먼저 보여주고, 이후 보완할 복구 구조는 별도 설계안으로 뒀습니다.

[단일 시스템 사례](#)[PRODUCT](#)[BACKEND](#)[OPERATIONS](#)[실제 구현 + 미구현 후속 설계](#)

상황 · 예약 생성 전에 외부 결제가 시작되고 환불 완료는 비동기 event로 도착하는 Memento AI 예약·결제 backend 사례입니다.

문제

결제 요청과 예약 처리가 서로 다른 시스템에서 진행돼 일부 단계만 성공할 수 있었습니다. 환불 요청 시 내부 자산을 먼저 바꾸면 외부 결제가 완료되기 전에 내부 상태가 앞서는 문제도 있었습니다.

대처 방법

- local transaction ID를 provider metadata에 넣어 내부 결제 이력과 Checkout·Webhook event를 연결했습니다.
- 예약 처리 실패 시 PaymentIntent 상태에 따라 cancel 또는 refund하는 provider-side 보상 흐름을 구성했습니다.
- 환불 요청과 완료를 분리하고 마일리지·이용권 변경을 완료 transition 뒤로 옮겼습니다.

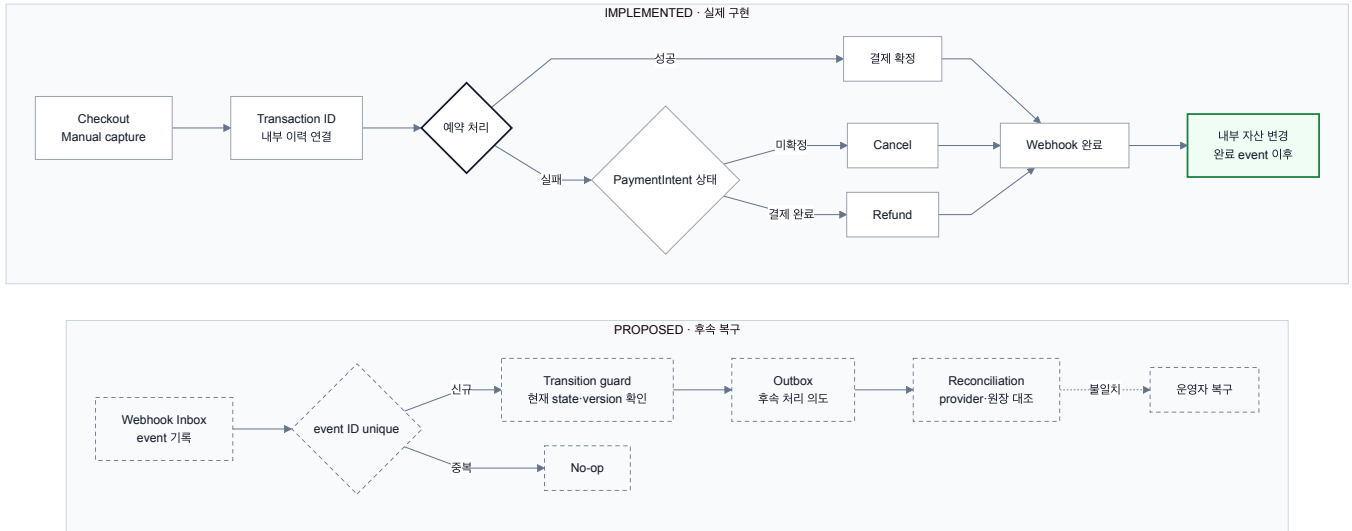
결과

선결제 slice는 구축을 주도했고 공유 결제 domain의 상태 정합성은 담당 범위에서 보완했습니다. Inbox·Outbox·reconciliation은 이 경험을 바탕으로 정리한 미구현 후속 설계안입니다.

결제 상태 흐름과 후속 복구 설계

설계 흐름

아래 그림은 실제 구현을 과장하지 않고, 동일한 금융 결제 문제를 다시 맡을 때 추가할 상태·복구 경계를 보여주는 PROPOSED 구조입니다.



위쪽은 실제 구현 범위이며, 아래쪽은 외부 결제와 내부 DB의 불일치를 더 체계적으로 복구하기 위한 미구현 후속 설계입니다.

기여 내용

해결 과정에서 맡은 기술 판단과 구현 범위입니다.

IMPLEMENTED · 실제 구현

Payment correlation

선택

local transaction ID로 PaymentHistory·PaymentMethod와 Checkout·Webhook event를 연결했습니다.

Provider compensation

선택

예약 처리 실패 시 PaymentIntent 상태에 따라 cancel/refund하는 보상 흐름을 구성했습니다.

Refund transition

선택

환불 요청과 완료를 분리하고 내부 자산 변경을 완료 event 이후로 옮겼습니다.

OBSERVED · 확인한 제약

내부 DB와 외부 결제 시스템은 하나의 atomic transaction으로 묶을 수 없습니다.

Webhook은 중복·지연·역순 도착할 수 있고 API 성공 응답이 최종 결제 완료를 뜻하지 않습니다.

Webhook Inbox

후속 설계안

provider event ID를 unique key로 저장해 중복 처리를 막고 처리 결과와 실패 원인을 남깁니다.

Transition guard

후속 설계안

현재 state-version을 확인해 늦게 도착한 event가 완료된 결제 상태를 되돌리지 못하게 합니다.

Outbox-reconciliation

후속 설계안

내부 상태 변경과 후속 처리 의도를 함께 기록하고 provider 상태와 내부 원장을 주기적으로 대조합니다.

TRADE-OFF · 도입 조건

Inbox·Outbox와 대조 작업은 운영 복잡도를 늘립니다. 결제량·불일치 빈도·수동 복구 비용이 단순 상태 처리보다 커질 때 단계적으로 도입합니다.

담당 범위

리드 **Stripe manual-capture 선결제**

Stripe Checkout 선결제 영역을 구축하고 local transaction ID로 내부 이력과 provider event를 연결했습니다.

기여 **취소·환불 상태 전이**

예약 실패 보상과 환불 요청·완료 분리, 내부 자산 변경 순서를 담당 범위에서 보완했습니다.

WHATSSUB JD FIT

금융·결제 data를 API와 상태 흐름으로 연결

외부 provider와 내부 transaction 사이의 부분 실패 처리

구현 경험을 바탕으로 복구 가능한 구조를 설계

실제 구현은 Checkout·Webhook·취소·환불이며, Inbox·Outbox·reconciliation은 후속 설계안입니다.

중간 전사와 확정 문맥이 충돌하지 않도록 상담 backend를 설계했습니다

STT partial·final·보정 event의 순서와 WebSocket session·비동기 task lifecycle을 함께 설계했습니다.

단일 시스템 사례

BACKEND

AI

OPERATIONS

운영 중 · 핵심 설계·통합 공동 수행

상황 · 상담 음성을 실시간 전사하고 대화 중 조언을 제공한 뒤, 상담 종료 후 문맥을 분석하는 AI 상담 backend 사례입니다.

문제

partial transcript는 빠르지만 뒤늦게 보정될 수 있고, final transcript와 다른 순서로 도착할 수 있습니다. 중복 event와 재연결 timer가 남으면 이전 turn의 작업이 새 문맥을 덮거나 종료된 session이 되살아날 수 있었습니다.

대처 방법

- DELTA·COMPLETE·optional CORRECTED를 같은 sequence에 묶어 늦은 보정 결과가 다른 turn을 덮지 않게 했습니다.
- DELTA는 domain keyword 조기 판단, COMPLETE는 확정 context·저장·LLM 판단으로 책임을 분리했습니다.
- 중복 event는 debounce하고 기존 task를 cancel·재생성하며 retry·turn-state guard로 비동기 충돌을 제어했습니다.
- stop guard와 timer·timeout·GC·shutdown 정리 경계를 보강해 종료된 session의 재연결을 차단했습니다.

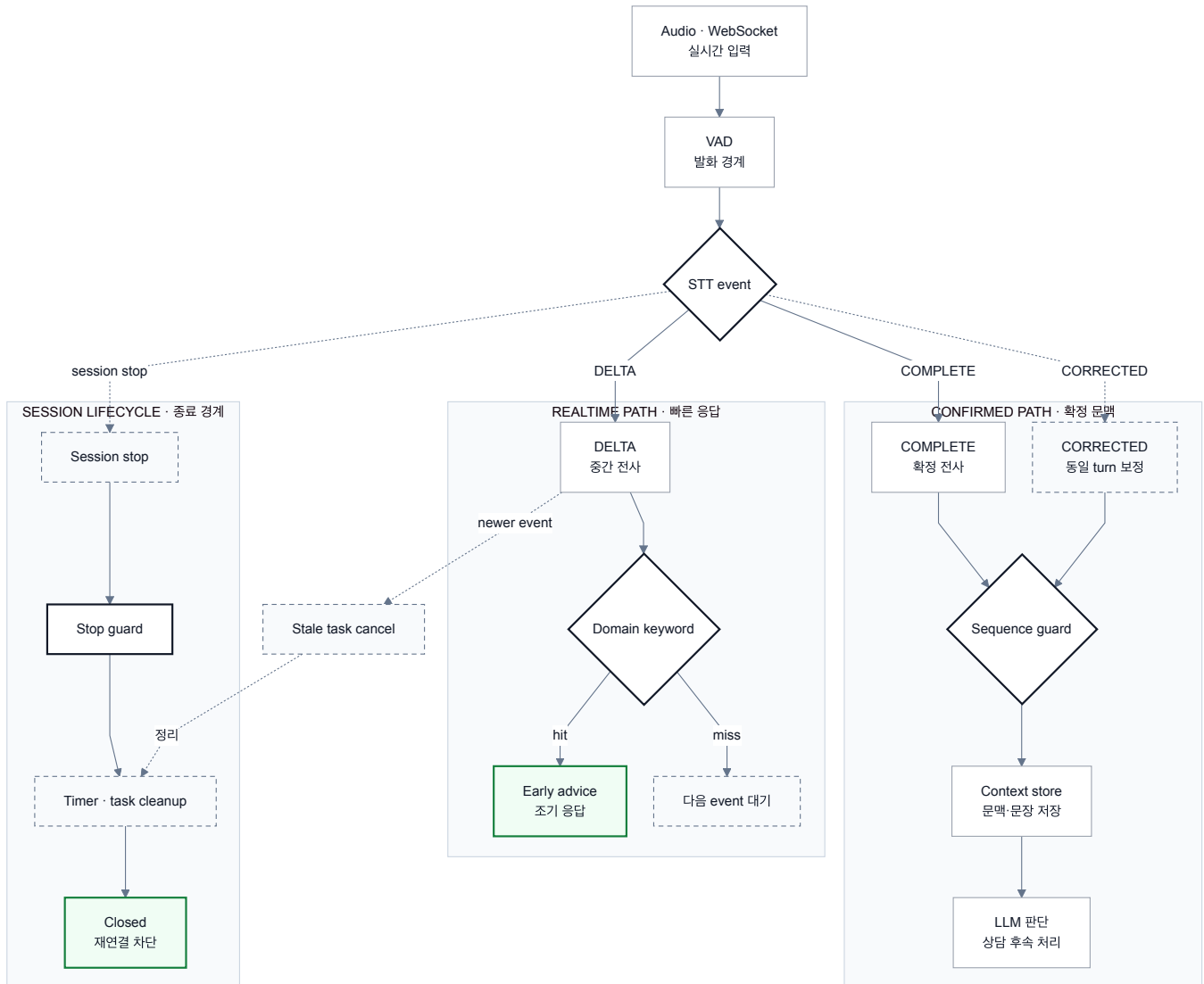
결과

실제 상담 E2E에서 transcript sequence의 누락·중복 여부와 advice trigger를 확인하고, 종료·재연결 경계는 focused regression scenario로 고정했습니다.

Realtime transcript-session lifecycle

설계 흐름

DELTA·COMPLETE·CORRECTED가 한 session에서 같은 turn을 가리키도록 sequence 기준을 정리했습니다.



중간·확정·보정 전사를 서로 다른 경로로 처리하고, sequence와 session 상태를 확인해 이전 결과가 새 문맥을 덮지 않게 했습니다.

기여 내용

해결 과정에서 많은 기술 판단과 구현 범위입니다.

EVENT ORDERING

DELTA·COMPLETE·CORRECTED를 같은 sequence로 묶어 늦은 보정이 다른 turn을 덮지 않게 했습니다.

FAST · CONFIRMED PATHS

DELTA는 domain keyword 기반 조기 advice trigger, COMPLETE는 확정 context·저장·LLM 판단에 사용했습니다.

TASK COORDINATION

중복 event를 debounce하고 기존 task를 cancel·재생성하며 retry·turn-state guard로 이전 작업의 stale write를 막았습니다.

SESSION CLEANUP

stop guard·timeout·GC·shutdown에서 timer와 연결을 정리해 종료된 session의 재연결을 차단했습니다.

담당 범위

기여 Session lifecycle·transcript event

WebSocket session의 event ordering·task coordination·cleanup 경계를 설계·통합하고 provider evaluation과 regression을 함께 수행했습니다.

WHATSSUB JD FIT

실시간 AI 기능을 session·event·비동기 task 경계가 있는 backend로 구현

Partial·final·corrected data의 ordering과 stale write 제어

Provider 평가와 회귀 scenario를 포함한 production 검증

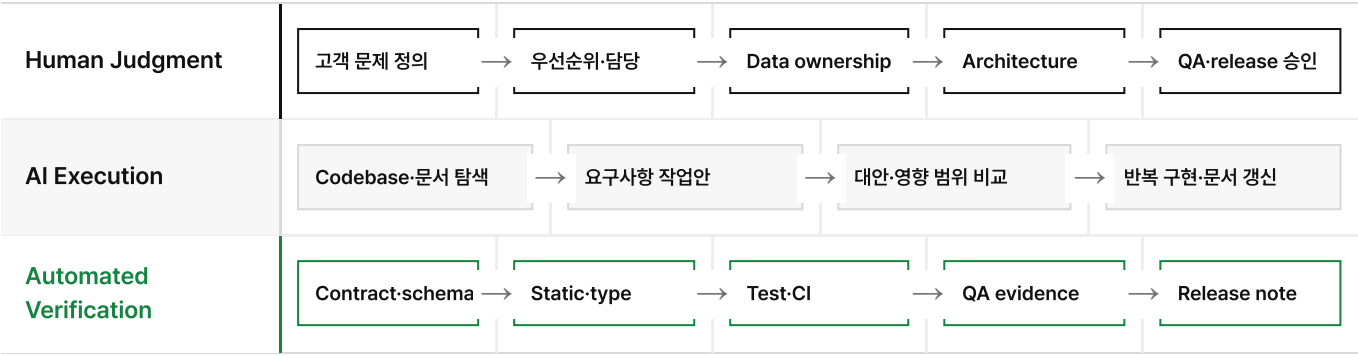
금융 도메인은 아니지만, 실시간 데이터와 비동기 AI 작업의 순서·복구를 운영 환경에서 다뤘습니다.

AI를 개인 도구가 아닌 팀의 제품 개발 체계로 확장했습니다

기획·디자인·개발·QA·release의 결정과 진행 상태를 하나의 실행 맥락에 쌓아, 사람과 AI가 같은 제품·architecture 기준에서 일하도록 했습니다.

공통 FastAPI template·agent context·자동 검증·release gate를 연결해 반복 설명과 인수인계에 드는 비용을 낮추고, 유사 기능과 후속 변경이 같은 기준에서 출발하게 했습니다.

SHARED CONTEXT	Decision-SPEC	→ Work Package-owner lane	→ FastAPI template·ADR·runbook	→ Test·QA evidence·release gate
----------------	---------------	---------------------------	--------------------------------	---------------------------------



AI가 만든 결과를 contract·test·실측 evidence로 걸러냈습니다

형식과 회귀 조건은 자동으로 확인하고, 실측 결과와 로그를 대조한 뒤 제품 품질과 release 여부는 사람이 판단합니다.

자동 게이트01	실측 분포 대조02	사람 판정03
틀리지 않았는가	기준에 부합하는가	실제로 통하는가
형식·schema	실측 결과 대조	제품 품질
정적 규칙	분포·로그 확인	예외 판단
회귀 조건	기준 재점검	Release decision

HUMAN GATE 자동 검증은 제품의 정답이 아닙니다. 최종 release 여부는 사람이 판단합니다.

실제 적용 범위	제품 개발 운영 Decision→release context	기획·디자인·개발·QA·release에서 나온 결정과 상태를 실행 맥락으로 누적하고 제품별 운영에 적용했습니다.
	Thready AI-assisted rebuild·quality system	backend 재구축과 생성 품질 system에서 architecture·검증 기준·cutover 판단과 AI의 실행 범위를 분리했습니다.
	Backend Template Shared architecture·agent context	팀과 agent가 같은 architecture·작업·검증 기준을 읽도록 공통 backend foundation과 context를 구축했습니다.

제품 판단이 실행 중 유실되지 않도록 결정→작업→검증→release 근거를 연결하고, AI는 그 경계 안에서 탐색과 반복 구현을 맡습니다.